PROJECT DOCUMENTATION

SkillSwap

Skill Sharing and Learning Management System

PREPARED BY Tanishka Anand Patil

AF CODE AF05319997

VERSION 1.0

BUILT WITH Laravel · MySQL · PHP

Document Control

Item	Details
Project	SkillSwap – Skill Sharing and Learning Management System
Prepared by	Tanishka Anand Patil
AF Code	AF05319997
Version	1.0
Purpose	Portfolio and technical project documentation
Status	Under development; current interface uses basic HTML in Laravel Blade templates
Evidence	Source code, route list, MySQL column checks and foreign-key query shared during review

Table of Contents

1. Project Overview 5
1.1 Introduction 5
1.2 Problem Statement 5
1.3 Objectives 5
1.4 Scope and Users 5
2. Requirements and Scope 6
2.1 Functional Requirements 6
2.2 Non-Functional Requirements 6
2.3 Assumptions and Unverified Scope 6
3. Technology Stack 7
3.1 Development Environment 7
3.2 Stack Rationale 7
4. System Architecture 8
4.1 Architectural Style 8
4.2 Request Flow 8
4.3 MVC Components 8
4.4 Architecture Flow Summary 8
5. Database Design 9
5.1 Users 9
5.2 Skills 9
5.3 User_Skills 9
5.4 Bookings 10
5.5 Messages 10
5.6 Reviews 10
5.7 Verified Foreign Keys 11
5.8 Relationship Summary 11
6. Module Description 12
6.1 Authentication 12
6.2 Skill Management 12
6.3 Teaching and Learning Preferences 12
6.4 Bookings 12
6.5 Booking Status 12
6.6 Messaging 12
6.7 Profiles 12
6.8 Reviews and Ratings 12
6.9 Average Rating 13
7. Project Folder Structure 14
7.1 Folder Responsibilities 14
8. Development Methodology 16
8.1 Version Control 16
8.2 Migration Consistency Note 16
9. Validation and Security 17
9.1 Authentication 17
9.2 Route Protection 17
9.3 Booking Controls 17
9.4 Messaging Controls 17

9.5 Review Controls 17
9.6 Further Security Tests 17
10. Testing Plan 19
10.1 Test Evidence 19
11. Installation and Execution 20
11.1 Prerequisites 20
11.2 Setup Steps 20
11.3 Useful Commands 20
11.4 Environment Safety 20
12. Current Limitations 21
12.1 Data-Model Considerations 21
13. Future Enhancements 22
14. Conclusion 23
15. References 24

1. Project Overview

1.1 Introduction

SkillSwap is a web application concept that connects users around skills they can teach and skills they want to learn. Users can record teaching and learning preferences, request sessions, communicate through messages, view profiles, and submit reviews for completed bookings. The application is being developed with Laravel and a MySQL database.

1.2 Problem Statement

People need a straightforward way to find someone who can teach a skill, arrange a session, and communicate with the other participant. Informal arrangements can leave preferences, schedules, booking status and feedback scattered across different channels. SkillSwap brings these activities into one application workflow.

1.3 Objectives

- Allow users to register, log in and log out.
- Maintain a catalogue of skills with names and optional descriptions.
- Let users mark skills as teach or learn.
- Allow learners to request sessions with users who offer the selected skill.
- Support booking status changes from pending to accepted/rejected and accepted to completed.
- Provide messaging with read timestamps and message edit/delete tracking.
- Allow reviews for completed bookings.
- Store application data in a relational database.

1.4 Scope and Users

The current scope includes authentication, skill browsing and creation, user skill preferences, bookings, messaging, profiles and reviews. The current interface is basic HTML rendered through Blade templates; advanced styling is not yet complete.

- Registered user: manages preferences, bookings and messages through available workflows.
- Learner: participant who requests a booking.
- Teacher: participant who offers a skill and responds to booking requests.

The inspected users table does not contain a role column. Learner and teacher describe a user's role in a particular booking, not a verified global account role.

2. Requirements and Scope

2.1 Functional Requirements

ID	Requirement	Implementation evidence
FR-01	Register	AuthController validates name, email, password and confirmation.
FR-02	Login/logout	Laravel Auth and session management in AuthController.
FR-03	Browse/add skills	SkillController lists and validates skill creation.
FR-04	Set teach/learn preferences	UserSkillController accepts teach or learn.
FR-05	Create bookings	BookingController validates teacher, skill, date, duration and message.
FR-06	Update booking status	Teacher can accept/reject pending booking or complete accepted booking.
FR-07	Messaging	MessageController supports conversations and message sending.
FR-08	Edit/delete messages	Controller checks participants and tracks deletion timestamps.
FR-09	View profiles	Profile route and Blade view exist; detailed controller behavior not reviewed.
FR-10	Review booking	ReviewController validates completed booking, participant and rating.
FR-11	Calculate ratings	User model calculates average received rating and review count.

2.2 Non-Functional Requirements

- Security: protected application routes use Laravel auth middleware.
- Integrity: verified foreign keys connect core tables.
- Maintainability: responsibilities are separated among routes, controllers, models and views.
- Validation: inspected controllers apply server-side validation.
- Usability: clear navigation and feedback are intended; current styling remains basic.
- Portability: application can run in a configured PHP/Laravel environment with a database.

2.3 Assumptions and Unverified Scope

- A MySQL database is configured using environment settings.
- Teachers are identified by a user_skills record with type='teach' for the selected skill.
- Only completed bookings can be reviewed through the inspected controller.
- Payments, video calls, notifications, calendar integration and admin roles are not verified.
- The database includes cancelled as a booking status, but the inspected status controller does not permit setting it.
- Email verification enforcement, rate limiting and automated test execution have not been verified.

3. Technology Stack

Technology/tool	Role	Project detail
Laravel	PHP web framework	Version reported during setup: 13.35.0
PHP	Server-side language	Version reported: 8.5.9
MySQL	Relational database	Database verified: skillswap
Composer	PHP dependency manager	Version reported: 2.10.3
Node.js / npm	JavaScript tooling	Versions reported: 26.11.1 / 11.20.0
Blade	Server-rendered templates	Views under resources/views
Eloquent ORM	Models and relationships	Six main models inspected
Git	Version control	Repository initialized and initial commit reported
MySQL Workbench	Database inspection	Used to inspect columns and foreign keys
PowerShell	Command line	Used for Artisan and source inspection

3.1 Development Environment

Project directory: C:\Users\DELL\Documents\skillswap. Local development URL: http://127.0.0.1:8000.

3.2 Stack Rationale

Laravel provides routing, validation, sessions, Eloquent models, migrations and Blade rendering. MySQL stores relational records. Composer manages PHP packages, while Node.js/npm are available for frontend tooling.

4. System Architecture

4.1 Architectural Style

SkillSwap follows Laravel's Model–View–Controller (MVC) organization. Routes match browser requests; controllers validate input and coordinate application logic; Eloquent models interact with MySQL; Blade templates render HTML.

4.2 Request Flow

4.3 MVC Components

Layer	Examples	Responsibility
Routes	routes/web.php	Maps HTTP methods and URLs to handlers.
Controllers	AuthController, BookingController, MessageController	Process requests, validate inputs, authorize actions and return responses.
Models	User, Skill, UserSkill, Booking, Message, Review	Represent data and define relationships.
Views	auth, bookings, messages, profiles	Render HTML forms and pages.
Database	MySQL skillswap	Persists records and foreign-key relationships.

4.4 Architecture Flow Summary

Step	Component	Interaction
1	Browser	Sends GET/POST/PATCH/DELETE request
2	Router + middleware	Matches route and checks authentication if required
3	Controller	Validates request and enforces application rules
4	Eloquent model	Queries or updates database records
5	MySQL	Returns records or confirms writes
6	Controller + Blade	Builds HTML response or redirect
7	Browser	Displays response

5. Database Design

Column definitions below are based on SHOW COLUMNS results shared from the skillswap database. Foreign keys are based on the INFORMATION_SCHEMA query. Nullable timestamps are marked NULL.

5.1 Users

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
name	VARCHAR(255)	Required name
email	VARCHAR(255)	Required; unique
email_verified_at	TIMESTAMP NULL	Verification timestamp field
password	VARCHAR(255)	Password hash
remember_token	VARCHAR(100) NULL	Remember-me token
created_at	TIMESTAMP NULL	Creation timestamp
updated_at	TIMESTAMP NULL	Update timestamp

5.2 Skills

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
name	VARCHAR(255)	Required skill name
description	TEXT NULL	Optional description
created_at	TIMESTAMP NULL	Creation timestamp
updated_at	TIMESTAMP NULL	Update timestamp

5.3 User_Skills

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
user_id	BIGINT UNSIGNED	FK to users.id
skill_id	BIGINT UNSIGNED	FK to skills.id
type	ENUM('teach','learn')	Preference type
created_at	TIMESTAMP NULL	Creation timestamp
updated_at	TIMESTAMP NULL	Update timestamp

5.4 Bookings

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
learner_id	BIGINT UNSIGNED	FK to users.id
teacher_id	BIGINT UNSIGNED	FK to users.id
skill_id	BIGINT UNSIGNED	FK to skills.id
scheduled_at	DATETIME	Required session date/time
duration_minutes	INT UNSIGNED	Default 60
message	TEXT NULL	Optional booking message
status	ENUM('pending','accepted','rejected','completed','cancelled')	Default pending
created_at	TIMESTAMP NULL	Creation timestamp
updated_at	TIMESTAMP NULL	Update timestamp

5.5 Messages

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
sender_id	BIGINT UNSIGNED	FK to users.id
receiver_id	BIGINT UNSIGNED	FK to users.id
message	TEXT	Required message content
read_at	TIMESTAMP NULL	Read timestamp
created_at	TIMESTAMP NULL	Creation timestamp
updated_at	TIMESTAMP NULL	Update timestamp
edited_at	TIMESTAMP NULL	Edit timestamp
deleted_for_sender_at	TIMESTAMP NULL	Sender-specific deletion timestamp
deleted_for_receiver_at	TIMESTAMP NULL	Receiver-specific deletion timestamp

5.6 Reviews

Column	Type	Constraint / purpose
id	BIGINT UNSIGNED	Primary key; auto increment
booking_id	BIGINT UNSIGNED	FK to bookings.id
reviewer_id	BIGINT UNSIGNED	FK to users.id
reviewee_id	BIGINT UNSIGNED	FK to users.id

Column	Type	Constraint / purpose
rating	TINYINT UNSIGNED	Controller validates 1–5
comment	TEXT NULL	Optional feedback
created_at	TIMESTAMP NULL	Creation timestamp
updated_at	TIMESTAMP NULL	Update timestamp

5.7 Verified Foreign Keys

Source column	Referenced column	Meaning
bookings.learner_id	users.id	Learner in booking
bookings.teacher_id	users.id	Teacher in booking
bookings.skill_id	skills.id	Skill for booking
messages.sender_id	users.id	Message sender
messages.receiver_id	users.id	Message receiver
reviews.booking_id	bookings.id	Booking reviewed
reviews.reviewer_id	users.id	Review author
reviews.reviewee_id	users.id	Review recipient
user_skills.skill_id	skills.id	Selected skill
user_skills.user_id	users.id	User selecting skill

5.8 Relationship Summary

- A user can have multiple user_skills records; each record references one user and one skill.
- A skill can be associated with multiple user_skills records and bookings.
- A booking references a learner, a teacher and a skill.
- A message references sender and receiver users.
- A review references one booking and two user records: reviewer and reviewee.

The foreign-key query returned ten relationships. The inspected message migration filenames and contents were inconsistent, so the current database inspection is treated as the schema source of truth.

6. Module Description

6.1 Authentication

AuthController handles registration, login and logout. Registration validates name, email uniqueness, password length and confirmation. Passwords are hashed before saving. Successful registration/login regenerate the session; logout invalidates it and regenerates the CSRF token.

6.2 Skill Management

SkillController retrieves all skills and accepts skill creation. The name is required text up to 100 characters; description is optional text up to 1,000 characters.

6.3 Teaching and Learning Preferences

UserSkillController displays all skills and the logged-in user's selections. It validates that the skill exists and that type is teach or learn. firstOrCreate() avoids duplicate matching records through application logic.

6.4 Bookings

BookingController lists bookings where the current user is learner or teacher. A request requires an existing teacher and skill, a future schedule, duration of 15–240 minutes, and an optional message up to 1,000 characters. The selected teacher must offer the selected skill, and self-booking is rejected. New bookings start pending.

6.5 Booking Status

The assigned teacher can change pending to accepted or rejected. An accepted booking can become completed. Other transitions are rejected by the controller. Although cancelled exists in the database enum, the inspected status method does not allow setting it.

6.6 Messaging

MessageController lists users, builds conversations, calculates unread counts, displays conversations and marks visible incoming messages as read. Message text is required and limited to 2,000 characters. The sender can edit an eligible message. Deletion can be recorded for one participant or, by the original sender, for both participants.

6.7 Profiles

The route /users/{user} maps to ProfileController@show and the profile view exists. Detailed profile controller behavior was not reviewed, so additional behavior is not specified.

6.8 Reviews and Ratings

Reviews are allowed only for completed bookings and only for a participant. The other participant is the reviewee. Rating is an integer from 1 to 5; comment is optional and up to 2,000 characters. The controller checks for a previous review by the same reviewer for that booking.

6.9 Average Rating

The User model provides averageRating() and reviewsCount() based on received reviews. averageRating() returns zero when no ratings exist.

7. Project Folder Structure

```
skillswap/
├── app/
│   ├── Http/Controllers/
│   │   ├── AuthController.php
│   │   ├── BookingController.php
│   │   ├── MessageController.php
│   │   ├── ProfileController.php
│   │   ├── ReviewController.php
│   │   ├── SkillController.php
│   │   └── UserSkillController.php
│   └── Models/
│       ├── User.php
│       ├── Skill.php
│       ├── UserSkill.php
│       ├── Booking.php
│       ├── Message.php
│       └── Review.php
├── database/migrations/
├── resources/views/
│   ├── auth/login.blade.php
│   ├── auth/register.blade.php
│   ├── bookings/index.blade.php
│   ├── messages/conversations.blade.php
│   ├── messages/index.blade.php
│   ├── messages/users.blade.php
│   ├── profiles/show.blade.php
│   ├── Skills/index.blade.php
│   ├── user-skills/index.blade.php
│   ├── dashboard.blade.php
│   └── welcome.blade.php
├── routes/web.php
├── .env
├── artisan
└── composer.json
```

7.1 Folder Responsibilities

Path	Purpose
app/Http/Controllers	Request handling and workflows
app/Models	Eloquent models and relationships
resources/views	Blade templates rendering HTML
routes/web.php	Web routes and auth middleware group
database/migrations	Schema change scripts
.env	Environment-specific settings; secrets must not be shared
composer.json	PHP dependency and project configuration

8. Development Methodology

SkillSwap has been developed incrementally. The reviewed work shows modules being added and checked through local execution, Artisan commands, source inspection and MySQL Workbench.

8.1 Version Control

The project owner reported Git initialization and an initial commit. Recommended practice is to commit each coherent feature, exclude .env secrets from Git, and keep migration files aligned with the actual database schema.

8.2 Migration Consistency Note

The two message-related migration files had names that did not match their current contents: one contained add-column logic while the other had empty callbacks. The database was inspected directly and its current structure recorded. These files should be reconciled before using migrations to create a fresh database.

9. Validation and Security

9.1 Authentication

- Registration requires name, valid unique email, password of at least eight characters and confirmation.
- Registration explicitly hashes the password with Hash::make().
- Successful registration and login regenerate the session.
- Logout logs out the user, invalidates the session and regenerates the CSRF token.

9.2 Route Protection

routes/web.php places dashboard, skills, user skills, bookings, reviews, profiles, people, conversations and message routes inside auth middleware. Registration, login and the root redirect are outside the group. Logout separately uses auth.

9.3 Booking Controls

- Teacher and skill must exist.
- Schedule must be in the future; duration must be 15–240 minutes.
- Teacher must offer the selected skill; self-booking is rejected.
- Only the assigned teacher can change status.
- Allowed transitions are pending to accepted/rejected and accepted to completed.

9.4 Messaging Controls

- Message is required text up to 2,000 characters.
- Self-messaging is rejected.
- Editing checks sender identity and conversation membership.
- Deletion checks participant identity and the requested action.
- Only the original sender can delete for everyone.
- Separate timestamps track sender and receiver deletion visibility.

9.5 Review Controls

- Only completed bookings can be reviewed.
- Reviewer must be a participant in the booking.
- Rating is constrained to integer 1–5; comment is optional up to 2,000 characters.
- Controller checks for a duplicate review from the same reviewer for that booking.

9.6 Further Security Tests

- Verify CSRF tokens in state-changing forms.
- Test protected routes while logged out.
- Attempt editing/deleting another user's message.
- Attempt booking/review requests using unrelated user IDs.

- Review login rate limiting and brute-force protections.
- Check migration consistency and constraints on a fresh installation.

This section describes inspected code and recommended tests. It is not a penetration-test report and does not claim that every security test has passed.

10. Testing Plan

These are planned manual test cases based on inspected code. Actual results and status are Pending because formal test execution results were not supplied.

ID	Scenario	Expected result	Actual	Status
TC-01	Register with valid data	Account created; dashboard opens	Pending	Not run
TC-02	Register duplicate email	Validation rejects email	Pending	Not run
TC-03	Password shorter than 8 chars	Validation rejects password	Pending	Not run
TC-04	Login with valid credentials	Dashboard opens	Pending	Not run
TC-05	Login with invalid credentials	Error displayed	Pending	Not run
TC-06	Open protected route logged out	Access blocked	Pending	Not run
TC-07	Add valid skill	Skill stored	Pending	Not run
TC-08	Save teach/learn preference	Preference saved once	Pending	Not run
TC-09	Create booking with past date	Validation rejects request	Pending	Not run
TC-10	Book self as teacher	Request rejected	Pending	Not run
TC-11	Teacher does not offer skill	Request rejected	Pending	Not run
TC-12	Teacher accepts pending booking	Status becomes accepted	Pending	Not run
TC-13	Unauthorized user changes status	Request denied	Pending	Not run
TC-14	Send message over 2,000 chars	Validation rejects message	Pending	Not run
TC-15	Sender edits own message	Text and edited_at updated	Pending	Not run
TC-16	Receiver deletes for everyone	Request denied	Pending	Not run
TC-17	Review incomplete booking	Request denied	Pending	Not run
TC-18	Rating outside 1–5	Validation rejects rating	Pending	Not run
TC-19	Duplicate review by same reviewer	Duplicate rejected	Pending	Not run
TC-20	Open conversation with unread messages	Eligible incoming messages marked read	Pending	Not run

10.1 Test Evidence

When testing is performed, record the date, steps, observed output and pass/fail status. Avoid including real passwords or private user information in screenshots.

11. Installation and Execution

These steps describe the local workflow. Validate a clean installation after reconciling migration files.

11.1 Prerequisites

- Compatible PHP version
- Composer
- MySQL server and database client
- Node.js/npm if frontend assets require them
- Git (optional, recommended)

11.2 Setup Steps

11.3 Useful Commands

Command	Purpose
php artisan about	Shows environment/application information
php artisan route:list	Lists registered routes
php artisan migrate:status	Shows migration status
php artisan optimize:clear	Clears cached framework files
Get-ChildItem app/Models	Lists models in PowerShell
Get-ChildItem app/Http/Controllers	Lists controllers
SHOW COLUMNS FROM messages;	Inspects table columns
INFORMATION_SCHEMA.KEY_COLUMN_USAGE query	Inspects foreign-key relationships

11.4 Environment Safety

Do not publish .env values, database passwords, application keys or real account credentials. Keep secrets out of Git.

12. Current Limitations

- UI currently uses basic HTML in Blade templates; visual styling is unfinished.
- Production deployment has not been verified.
- Message migration filenames and contents are inconsistent and should be reconciled.
- The inspected booking status controller does not support cancellation, although the database enum includes cancelled.
- No global role column was found in the inspected users table.
- Formal test cases have not been executed; actual results remain pending.
- Detailed ProfileController behavior was not reviewed.
- Automated tests, email verification enforcement, rate limiting, notifications and payment processing are not confirmed.

12.1 Data-Model Considerations

UserSkillController uses firstOrCreate() to avoid duplicate matching preferences through application logic, but a database-level unique constraint was not verified. ReviewController checks for duplicate reviews in application logic, but a database-level unique constraint was not verified. Booking model declares reviews() as hasMany, while the controller prevents a reviewer from submitting a second review for the same booking.

13. Future Enhancements

- Add responsive styling and accessible forms.
- Improve dashboard layout for desktop and mobile.
- Add search and filtering for skills and teachers.
- Implement booking cancellation with clear rules and authorization.
- Add notifications for booking changes and new messages.
- Add pagination for growing lists.
- Expand automated tests for validation and authorization.
- Reconcile migration files for reproducible setup.
- Add database uniqueness constraints where business rules require them.
- Expand profile editing if required.
- Review email verification and login rate limiting.
- Document production deployment and environment configuration.

14. Conclusion

SkillSwap is a Laravel-based skill sharing and learning management application. The reviewed implementation includes authentication, a skills catalogue, teach/learn preferences, booking requests and status changes, one-to-one messaging, profiles and reviews. The application uses Eloquent models and a MySQL database with verified foreign keys connecting users, skills, bookings, messages and reviews.

The project demonstrates practical use of MVC separation, request validation, session handling, password hashing, relational database design and controller-level authorization checks. The interface remains in a basic HTML stage. Migration inconsistency and planned tests should be addressed before treating the project as ready for fresh installation or production.

Next priorities are to reconcile message migrations, test major workflows, complete UI styling and record actual test outcomes. This document provides a technical baseline that can be updated as the project progresses.

15. References

- Laravel Documentation — https://laravel.com/docs
- Laravel Eloquent ORM — https://laravel.com/docs/eloquent
- Laravel Validation — https://laravel.com/docs/validation
- Laravel Authentication — https://laravel.com/docs/authentication
- MySQL Reference Manual — https://dev.mysql.com/doc/
- PHP Documentation — https://www.php.net/docs.php
- Composer Documentation — https://getcomposer.org/doc/
- Node.js Documentation — https://nodejs.org/en/docs
- Project source files inspected: routes/web.php, app/Models, app/Http/Controllers and resources/views.
- Database verification performed through MySQL Workbench against the skillswap database.